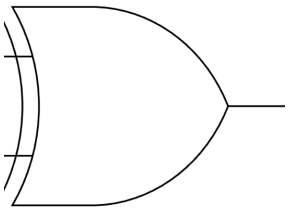
A 1.2.5 Logic Diagrams



1.2.5 Construct logic diagrams

This section further explores logic gates, the building blocks of digital circuits. Learn to construct and interpret logic diagrams, understand gate functions (AND, OR, NOT, etc.), and simplify logic expressions using Boolean algebra. Master the fundamentals of digital circuit design.

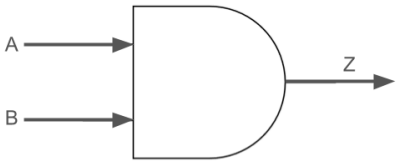
Key Questions

- How can logic diagrams demonstrate the connection and interaction of logic gates in a circuit?
- What standard symbols are used for AND, OR, NOT, NAND, NOR, XOR, and XNOR gates?
- How are inputs processed in logic diagrams to produce outputs?
- How can combinations of logic gates be used to perform more complex logical operations?
- How can Boolean algebra rules be applied to simplify complex logic diagrams and expressions?

Logic gate symbols

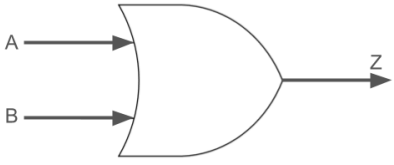
Here is a recap of all of the logic gates, their associated truth tables, and the symbols used for them.

AND



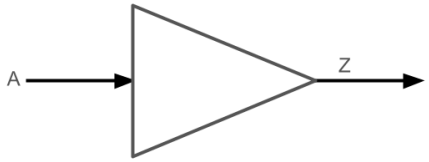
A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR



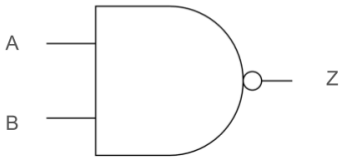
A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

NOT



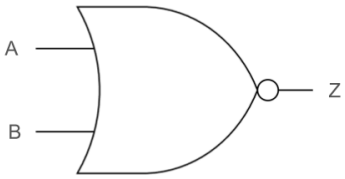
A	Z
0	1
1	0

NAND



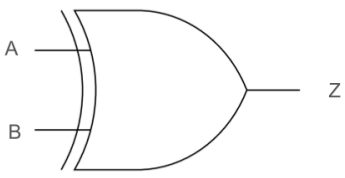
A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

NOR



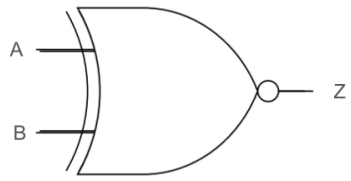
A	B	Z
0	0	1
0	1	0
1	0	0
1	1	0

XOR



A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

XNOR



A	B	Z
0	0	1
0	1	0
1	0	0
1	1	1

Drawing logic diagrams

You can be asked to draw a logic diagram to represent values in a truth table or a boolean expression.

How Inputs are Processed in Logic Diagrams

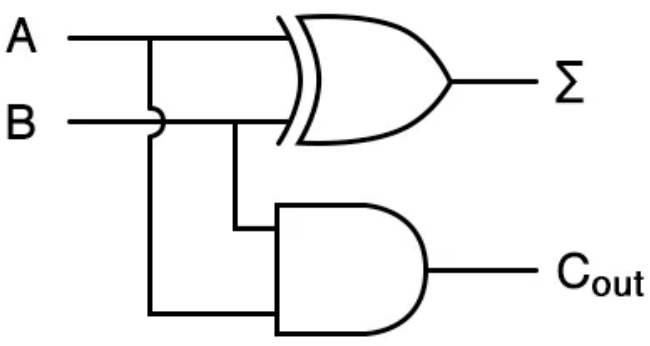
Inputs to a logic gate are represented by binary values, 0 or 1. The logic gate's output depends on the combination of its inputs and the gate's function. A truth table defines the relationship between inputs and outputs.

Combining Logic Gates for Complex Operations

Combining different logic gates allows us to create circuits that perform more complex logical operations. For example, we can build a circuit that adds two binary numbers or compares two values.

Adding two binary numbers

This is done with a circuit called a binary adder. The simplest version is a half adder, which adds two single-bit binary numbers. 1 Here's what it looks like:



Half Adder truth table

A	B	SUM	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Inputs: A and B (single bits)
Outputs: Sum (A XOR B) and Carry (A AND B)

Boolean Algebra and Logic Diagram Simplification

Boolean algebra is a mathematical system for manipulating logical expressions. It provides a set of rules for simplifying complex logic diagrams and expressions. By applying Boolean algebra rules, we can reduce the number of gates needed to implement a particular function, leading to more efficient circuits.

We have seen before that we can use algebraic statements to represent logic gates.
The order of operations for Boolean algebra, from highest to lowest priority, is NOT, then AND, then OR. Expressions inside brackets are always evaluated first.

Logic	Algebraic Expression
A AND B	A . B
A OR B	A + B
NOT A	$\bar{A}$

Laws of Boolean algebra

1. Annulment Law
A . 0 = 0 (Anything ANDed with 0 is 0)
A + 1 = 1 (Anything ORed with 1 is 1)
2. Identity Law
A . 1 = A (Anything ANDed with 1 is itself)
A + 0 = A (Anything ORed with 0 is itself)
3. Idempotent Law
A . A = A (A variable ANDed with itself is itself)
A + A = A (A variable ORed with itself is itself)
4. Complement Law
A . $\bar{A}$ = 0 (A variable ANDed with its complement is 0)
A + $\bar{A}$ = 1 (A variable ORed with its complement is 1)

5. Double Negation Law
$\bar{\bar{A}} = A$ (The NOT of NOT A is A)
6. Commutative Law
A . B = B . A (The order of ANDing doesn't matter)
A + B = B + A (The order of ORing doesn't matter)
7. Associative Law
A . (B . C) = (A . B) . C (Grouping in ANDing doesn't matter)
A + (B + C) = (A + B) + C (Grouping in ORing doesn't matter)
8. Distributive Law
A . (B + C) = (A . B) + (A . C) (AND distributes over OR)
A + (B . C) = (A + B) . (A + C) (OR distributes over AND)
9. De Morgan's Theorem
$\overline{(A . B)} = \bar{A} + \bar{B}$ (The NOT of an AND is the OR of the NOTs)
$\overline{(A + B)} = \bar{A} . \bar{B}$ (The NOT of an OR is the AND of the NOTs)
10. Absorption Law
A . (A + B) = A
A + (A . B) = A
11. Redundancy Law
(A + B) . (A + $\bar{B}$) = A
A . B + A . $\bar{B}$ = A

Using Boolean Algebra Laws

These laws are used to:

- **Simplify Boolean expressions:** Reduce the complexity of logical statements.
- **Manipulate logic circuits:** Transform circuits into equivalent forms, often with fewer gates.
- **Prove the equivalence of logic expressions:** Demonstrate that two different expressions represent the same logical behaviour.

Example 1

Let’s take the following expression taken from a logic diagram:
(A . B) + (A . $\overline{B}$) + C
This expression involves three variables (A, B and C), their complements, and both AND and OR operations. It requires a combination of Boolean algebra laws to simplify.
Here's how you can approach the simplification process, keeping in mind there might be different valid paths to the solution (Click the eye for the solution):

Example 2

Let’s take the following expression taken from a logic diagram:
(A + B) . (A . $\overline{C}$)
This expression involves three variables (A, B and C), their complements, and both AND and OR operations. It requires a combination of Boolean algebra laws to simplify.
Here's how you can approach the simplification process, keeping in mind there might be different valid paths to the solution (Click the eye for the solution):

Example 3

Let’s take the following expression taken from a logic diagram:
$(C + \bar{D}) \cdot (B \cdot C) + (B \cdot D)$
This expression involves three variables (B, C, and D) and a combination of AND and OR operations. Here's how you can approach the simplification process, keeping in mind there might be different valid paths to the solution (Click the eye for the solution):